

# Detailed PredictionService Architecture & Industrial Logic Report

This document explains not only WHAT PredictionService does, but also WHY each architectural decision was introduced and HOW the components interact together in the MDF1 industrial ML platform.

PredictionService is not simply a model inference API. It evolved into an industrial-grade online ML serving platform aligned with TrainingService governance, industrial-safe runtime validation, traceability, explainability, and future production deployment requirements.

## 1. Why PredictionService Exists

TrainingService performs offline ML orchestration:  
data cleaning,  
feature engineering,  
model training,  
evaluation,  
and artifact publishing.

However, industrial production systems cannot directly use training notebooks or raw ML code.

PredictionService exists to safely consume trained artifacts and expose them through stable APIs.

Its responsibilities are:

- loading the correct production-ready models,
- validating incoming industrial data,
- enforcing feature contracts,
- generating predictions,
- producing explainability,
- logging runtime behavior,
- and providing traceable outputs to GUIService and future MES integrations.

This separation is extremely important in enterprise architecture because: training environments are unstable and experimental, while prediction environments must be stable, reproducible, and safe.

## 2. Registry-Driven Serving Philosophy

Originally models were loaded using hardcoded logic.

This caused several industrial problems:

- difficult deployment management,
- duplicated serving logic,
- risky manual switching,

- inability to manage ensembles cleanly,
- and weak traceability.

The architecture evolved toward:  
best\_models\_registry.json

This registry acts as the serving contract between:  
TrainingService → PredictionService.

The registry decides:

- which target uses which model,
- whether serving is single\_model or weighted\_ensemble,
- ensemble member mappings,
- ensemble weights,
- and artifact locations.

Why is this important?

Because PredictionService should NOT decide business deployment logic itself.  
TrainingService evaluates and selects the best production candidates.  
PredictionService only enforces and serves those decisions safely.

### 3. Artifact Governance

Each model is published as a controlled artifact bundle.

Example:  
artifacts//

Containing:

- model.joblib
- features.json
- metrics.json
- metadata.json

Why separate artifacts?

model.joblib:  
contains the trained sklearn pipeline and estimator.

features.json:  
contains the exact training feature order.  
This is critical because wrong feature ordering can completely corrupt predictions.

metrics.json:  
stores evaluation metrics such as:  
MAE,  
RMSE,  
R<sup>2</sup>,  
validation performance.

metadata.json:  
stores algorithm metadata,  
training information,

and future explainability/configuration metadata.

PredictionService validates these artifacts before serving.  
This prevents corrupted deployments and incomplete production releases.

## 4. Exact Feature-Order Enforcement

One of the biggest industrial risks in ML serving is feature mismatch.

If incoming features are shuffled,  
predictions may still run,  
but outputs become mathematically wrong.

PredictionService now rebuilds prediction DataFrames strictly using:  
features.json

This guarantees:

- exact training order,
- consistent preprocessing,
- identical runtime structure.

Why is this critical?

Industrial operators may unknowingly:

- rename fields,
- reorder payloads,
- add extra variables,
- remove required process signals.

Without strict governance,  
predictions may silently become invalid.

## 5. Industrial-Safe Validation

PredictionService now validates incoming runtime payloads before prediction.

Validation includes:

- missing features,
- extra features,
- invalid numeric values,
- NaN values,
- forced production features.

Why?

Industrial data is noisy.  
MES and KSoft exports may contain:

- broken values,
- empty strings,
- malformed objects,
- incomplete process records.

The validation layer acts as a runtime industrial safety gate.

Instead of allowing hidden corruption,  
PredictionService now explicitly detects and reports problematic inputs.

## 6. Weighted Ensemble Serving

The architecture now supports:

single\_model  
and  
weighted\_ensemble.

Why ensembles?

Different algorithms capture different process behaviors.

Example:

Random Forest:

- stable,
- robust,
- nonlinear,
- resistant to noise.

XGBoost:

- strong optimization,
- high predictive power,
- strong feature interaction learning.

The ensemble layer combines strengths of multiple models.

PredictionService:

- predicts each member,
- applies configured weights,
- computes weighted averages,
- returns member-level traceability.

This improves:

- industrial robustness,
- stability,
- generalization,
- production resilience.

## 7. SHAP Explainability

PredictionService does not only return predictions.

It also explains WHY predictions happened.

The architecture uses:

SHAP explainability.

SHAP computes feature contribution values for each prediction.

This provides:

- local prediction reasoning,
- operator insight,
- process understanding,
- debugging capability.

Outputs include:

- feature name,
- value,
- contribution impact,
- positive/negative direction,
- absolute importance.

Why is this important industrially?

Operators do not trust black-box systems.

Explainability bridges:

ML systems ↔ process engineers.

## 8. Recommendation Layer

Raw SHAP values are difficult for operators to interpret.

Therefore:

recommendation\_engine.py converts explainability into:  
human-readable operational guidance.

Example:

'press pressure is increasing predicted density'

This allows:

- easier operational interpretation,
- process-awareness,
- future recommendation systems,
- future advisory control logic.

The architecture intentionally separates:

prediction  
from  
human interpretation.

## 9. Risk Evaluation Layer

Prediction values alone are not enough operationally.

PredictionService now converts predictions into:

LOW

MEDIUM  
HIGH  
risk levels.

Why?

Operators and dashboards require:  
simple operational interpretation.

Example:  
labDensityAverage = 820

This number alone has little operational meaning.

But:  
HIGH RISK

immediately communicates urgency.

This layer transforms ML output into business-operational language.

## 10. Database Traceability

PredictionService logs prediction events into PostgreSQL.

Why?

Industrial AI systems require:

- traceability,
- auditability,
- historical investigation,
- future drift analysis,
- operational monitoring.

Stored information includes:

- target,
- prediction,
- model,
- serving type,
- feature diagnostics,
- risk levels,
- production order,
- board ID,
- timestamps.

This allows future:

- dashboards,
- analytics,
- KPI monitoring,
- process investigations,
- root-cause analysis.

## 11. Centralized Logging & Decorators

Originally logging behavior was fragmented.

The service was refactored using:

logging\_utils.py  
decorators.py

Why?

Centralized logging improves:

- debugging,
- maintainability,
- execution tracing,
- runtime monitoring.

Decorators automatically track:

STARTED,  
FINISHED,  
FAILED,  
execution duration.

This creates cleaner service orchestration  
without duplicating logging logic everywhere.

## 12. Final Architectural Position

PredictionService is no longer a simple API around sklearn models.

It now behaves as:  
an industrial online ML serving platform.

It includes:

- artifact governance,
- registry-driven serving,
- ensemble orchestration,
- industrial-safe validation,
- explainability,
- recommendations,
- traceability,
- centralized configuration,
- modular architecture.

Final architecture:

TrainingService



publishes artifacts + registry



PredictionService



serves validated predictions



GUIService / MES / dashboards

This architecture now resembles enterprise industrial AI platform design, rather than experimental notebook ML workflows.